1. Title of the Project

AI-Assisted Software Metrics Advisor with Intelligent Project Health Monitoring

2. Abstract

Software projects are often reviewed for functionality and delivery progress, while internal quality dimensions such as complexity, maintainability, coupling, cohesion, duplication, documentation, and testing practices can remain distributed across tools. Manually examining these dimensions across source files and tracking their changes between releases requires effort and makes project-wide interpretation difficult.

This project implements a web-based static software-quality analysis platform. A user uploads a project ZIP, which is validated and safely extracted before supported source files are scanned and parsed. The analysis engine calculates source metrics and produces category scores, an overall Project Health Score, and deterministic module-level risk assessments. An optional AI Advisor interprets stored metric and risk evidence and returns recommendations; it does not calculate or replace the deterministic scores. The React dashboard supports project and version selection, historical health trends, version comparison, Architecture Intelligence, Quality Gate and Language Health views. The application also generates PDF reports and exports supported dashboard visualizations as SVG.

The implementation uses Python, FastAPI, React, JavaScript, Vite, SQLite/SQLAlchemy, ReportLab, Radon, Javalang, and an optional configured AI provider. The code supports static analysis of Python, Java, C, C++, JavaScript, and TypeScript. Ollama integration and the Qwen 2.5 3B model are supported by configuration and tests, but the model is not hard-coded as the default and requires a separately running local Ollama installation. The key innovation is the integration of deterministic metrics, explainable module risks, AI-assisted interpretation, immutable project versions, file-level comparison, and historical monitoring in one application.

3. Problem Statement

Software teams need to assess code quality across multiple source files and across successive versions of the same project. Individual metrics can reveal specific concerns, but they do not by themselves provide a compact project-level view or a consistent way to prioritize risky modules. Manual comparisons also make it difficult to distinguish file additions, removals, modifications, and verified unchanged content.

The project addresses the need for a unified tool that statically analyzes uploaded source projects, summarizes measurable quality signals, identifies modules requiring attention using deterministic rules, provides evidence-grounded AI recommendations when configured, and retains version history for comparison. The analyzer does not execute uploaded source code and does not claim runtime test coverage.

4. Existing System

In a conventional workflow, developers may use code review, language-specific static-analysis tools, test tools, and manually maintained release notes. Metrics and risk information can be separated across these tools, while project-level health and version-to-version changes may require additional aggregation.

Limitations of the Existing System

- Software quality metrics are often handled independently, which can increase the effort required for analysis.
- An integrated view of overall project health and individual quality categories may be lacking.

- Determining which modules require immediate attention may require manual analysis of multiple metrics.
- Existing version comparison methods may not clearly identify new, deleted, modified, and unchanged files through hash verification.
- Monitoring changes in project health and module-level risks across different versions may not be straightforward.
- Converting analysis results into meaningful recommendations and well-structured reports may require additional manual effort.

These points describe the conventional workflow targeted by this project, not a claim that every existing commercial or open-source tool lacks these capabilities.

5. Proposed System

The proposed system provides one workflow from ZIP upload through static analysis, scoring, risk assessment, persistence, visualization, and reporting. Each successful upload creates a scan/version record. Project identity groups versions of the same logical project, so V1, V2, V3 and later uploads remain related rather than being treated as unrelated projects.

System Workflow

Stage	Description
Project ZIP upload and validation	Validate file type and enforce upload constraints.
Safe extraction and source scanning	Guard against unsafe archive paths and oversized archives; skip configured dependency/build directories.
Language-specific static analysis	Parse supported Python, Java, C, C++, JavaScript and TypeScript source files.
Metrics and scoring	Calculate supported metrics, normalize available values, derive category scores and overall health.
Deterministic risk analysis	Assign module risk score, level and named factors using explicit rules.
AI Advisor	Use structured stored evidence to explain issues and recommend actions; AI is not the scoring engine.
Persistence and versioning	Store scan data, version snapshots, file fingerprints and architecture results in SQLite.
Dashboard and outputs	Show project health, trends, comparisons, Architecture Intelligence, Quality Gate, Language Health, PDF and SVG exports.

6. Innovation

The innovation is the way several practical software-quality functions are integrated into a version-aware monitoring workflow, rather than the use of AI alone.

- **Unified multi-metric health view.** Supported metric values are normalized into category scores and an overall Project Health Score, with missing values kept unavailable rather than silently treated as zero.
- **Deterministic and explainable risk.** Module-level risk scores, risk levels, and risk factors are computed by rule-based logic independently of AI output.
- **Evidence-based AI interpretation.** The Advisor receives structured project metrics and risk context and is instructed not to recalculate scores or invent unavailable values.

- **Project identity and immutable versions.** Successful uploads are recorded as versions under a stable project identity, enabling V1-to-V2-to-V3 history.
- **File-level version comparison.** Stored fingerprints verify unchanged content where available; comparison identifies added, removed, changed and unchanged files and reports metric deltas and rule-based regression/improvement signals.
- **Practical code-churn proxy.** The implemented version-to-version signal uses added/removed file LOC and absolute LOC change for modified files; it does not claim Git commit statistics or exact changed-line diffs.
- **Architecture and decision support.** Architecture Intelligence, an advisory Quality Gate, Language Health comparison, dashboard trends, PDF reporting and SVG chart export provide additional ways to interpret results.

The contribution is a connected, inspectable workflow in which deterministic analysis remains authoritative, AI recommendations are a separate interpretation layer, and stored version evidence supports longitudinal comparison.

7. Objectives

- To validate and safely extract uploaded software-project ZIP files.
- To statically analyze supported source files and calculate software-quality metrics.
- To generate category scores and an overall Project Health Score from available metric values.
- To identify risky modules with deterministic scores, levels, and risk factors.
- To provide optional AI-assisted recommendations grounded in stored quality evidence.
- To preserve multiple versions of a logical project and display historical health trends.
- To compare versions at file and metric level and report regression/improvement signals.
- To implement Architecture Intelligence, Quality Gate and Language Health views.
- To generate PDF reports and export supported dashboard charts as SVG.
- To allow deletion of one selected logical project and its associated stored versions/data without deleting unrelated projects.

8. Methodology

1. **Problem identification:** Identify the difficulty of combining software-quality measurements and tracking quality across versions.
2. **Requirement and source review:** Review the existing project design and define supported metrics, languages, risk signals, history, comparison and reporting requirements.
3. **System design:** Separate frontend, REST API, source analysis, scoring, deterministic risk logic, AI provider, persistence, comparison and reporting components.
4. **Upload and scanning:** Validate the uploaded ZIP and apply safe extraction controls before discovering source files.
5. **Parsing and metric calculation:** Parse source structure with language-specific analyzers and calculate metrics supported by the source implementation.
6. **Scoring and risk analysis:** Normalize available metric values, calculate category/overall health, and apply deterministic module-risk rules.
7. **AI recommendation:** When configured, send structured quality data to the provider and normalize/ground its response; AI does not change stored deterministic scores.
8. **Persistence and versioning:** Save each scan and immutable version metadata, plus file fingerprints and architecture-analysis results.
9. **Comparison and monitoring:** Compare stored versions, identify file statuses and metric changes, calculate the implemented churn proxy, and display history trends.

10. **Evaluation:** Use the sample projects and test files included with the repository. Report only documented measurements; do not infer universal accuracy or performance from one observed run.

9. Hardware and Software Requirements

Hardware Requirements

Component	Requirement / status
Computer	Laptop or desktop capable of running the backend and frontend development tools; exact hardware model not specified in the project.
Memory	Not specified as a verified minimum in the implementation files.
Storage	Sufficient local storage for the application, SQLite database, uploaded ZIP processing and generated reports; exact amount not measured.
Local AI hardware	Depends on the selected Ollama model and local runtime; no model-specific hardware benchmark is supplied.

Software Requirements

Software / technology	Purpose / verified use
Python	Backend and static source-analysis pipeline.
FastAPI / Uvicorn	REST API and application server.
React / JavaScript / Vite	Dashboard interface and frontend build tooling.
SQLite / SQLAlchemy	Persistent project, scan/version, metric, risk and related records.
Radon	Python metric calculations, including complexity/maintainability support.
Javalang	Java source parsing.
Custom static parsers	C/C++, JavaScript and TypeScript structure analysis; these are not full compiler/type-resolution passes.
ReportLab	PDF report generation.
Ollama (optional)	Local LLM provider when explicitly configured and running.
Qwen 2.5 3B (optional example)	Model identifier supported in configuration/tests; not the application's fixed default and not bundled as an installed model.
HTML / CSS	Dashboard structure and presentation.

The source configuration defaults the AI provider to “none”. Ollama requires a separately installed/running service and a model pulled by the operator. The codebase includes automated test files, but this document does not claim that the complete suite passed during this document-generation task.

10. System Architecture

Figure 1 summarizes the implemented flow. The metrics, health scoring and module-risk components are deterministic. The AI Advisor is optional and interprets stored evidence. SQLite stores project identities, version snapshots, scan results, fingerprints and architecture-analysis data; the dashboard obtains results through the FastAPI API.

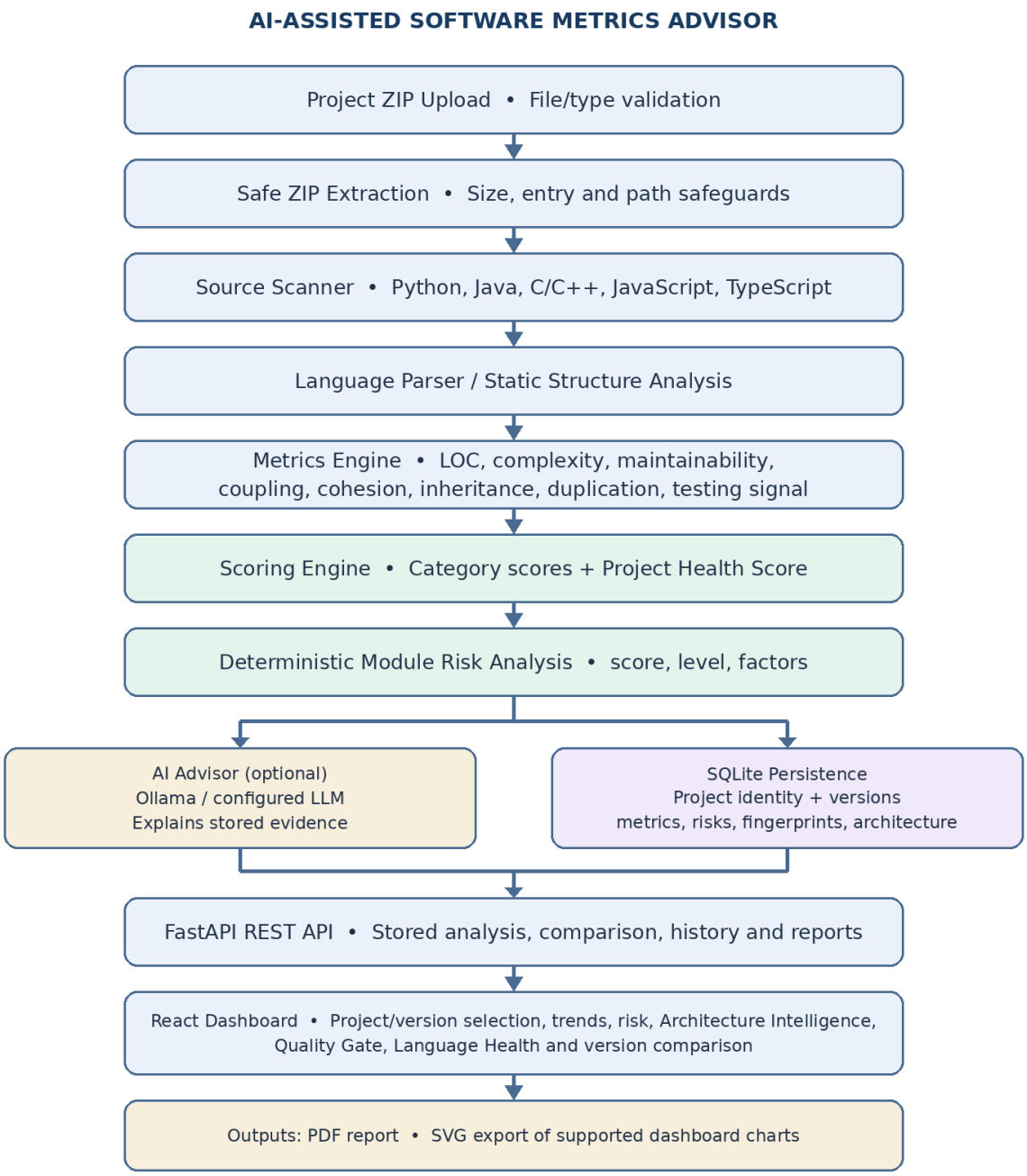


Figure 1. Architecture of the implemented AI-Assisted Software Metrics Advisor.

Key architectural boundaries

- Uploaded source code is scanned and parsed; it is not executed by the analyzer.
- Static parser support is implemented for Python, Java, C, C++, JavaScript and TypeScript. These parser paths do not provide full compiler-grade type resolution.
- The AI Advisor receives structured metric/risk information and does not compute health or risk scores.
- Version comparison uses persisted per-file metric values and content fingerprints; it does not rerun an earlier version's analysis.
- PDF and SVG are separate outputs: PDF reports are generated by the backend, while SVG export combines supported SVG charts already rendered in the dashboard.

11. Implementation

Module 1 – Project Upload and Safe Extraction

The upload API accepts a project ZIP and enforces configured upload-size, extraction-size and archive-entry limits. Safe extraction checks guard against path traversal and unsafe archive entries. Source scanning skips configured directories such as dependency, build and version-control output.

Module 2 – Source Scanner and Language Parsers

The scanner recognizes Python, Java, C, C++, JavaScript and TypeScript extensions. Python and Java use language-specific parsing paths; C/C++ and JavaScript/TypeScript use custom static lexing/structure analysis. The latter are pragmatic static analyzers, not full compilers.

Language	Implementation notes
Python	AST-based parsing; Radon is used for relevant metric calculations.
Java	Javalang parser and project-specific metric calculations.
C / C++	Custom lexer and structure scanner; metric approximations use extracted structure.
JavaScript / TypeScript	Custom lexer and structure scanner; JSX/TSX extensions are recognized.

Module 3 – Software Metrics Engine

Metric	How the implementation represents it / limitation
Lines of Code (LOC)	Tracks total, code, comment and blank lines using language-aware counting.
Cyclomatic Complexity	Calculates complexity from recognized control-flow constructs/functions; language parsing approach affects coverage.
Maintainability Index	0–100 index; Radon for Python and a project implementation for other supported parsers.
Coupling	Static import/reference-based dependency count. It is an approximation, not full CBO/type-resolution analysis.
Cohesion	Class-level LCOM1 based on method access to shared fields; null when insufficient evidence exists.
Depth of Inheritance	Project-wide class registry follows the recognized base-class chain; multiple inheritance is simplified.
Code Duplication	Detects repeated fixed-size line windows; it is not a semantic clone detector.
Documentation	Uses comment-to-code line ratio as a documentation proxy, not a review of documentation correctness.
Testing	Static test-file presence signal based on common test file/directory naming conventions; not line/branch coverage.
Code Churn	Version-to-version proxy based on file additions/removals and LOC deltas for modified files; unavailable for the first version.

Module 4 – Health Score and Category Scores

The scoring engine normalizes available metrics to a higher-is-better 0–100 scale, calculates category scores from available module-level values, and computes a weighted Project Health Score. The weights and normalization curves are project design choices, not a universal industry standard. Categories without available measurements are not silently assigned zero or 100; the overall weighting is renormalized over available categories.

Health score	Status
0–39	CRITICAL
40–59	POOR
60–74	NEEDS ATTENTION
75–89	HEALTHY
90–100	EXCELLENT

Module 5 – Deterministic Risk Analysis

A rule-based risk component calculates module risk scores and assigns LOW, MEDIUM, HIGH or CRITICAL levels with named risk factors. The risk assessment is deterministic and remains separate from the optional AI recommendations.

Module 6 – AI Advisor

The AI Advisor passes structured project health, category, metric and risk information to a configured provider. The provider integration supports a local Ollama chat endpoint and a configured hosted provider path. Responses are normalized into recommendation objects and guarded against unavailable providers, malformed responses and unsupported values. The system prompt instructs the model to treat stored scores as authoritative and to preserve the distinction between zero and unavailable (null) values. The default configuration disables AI until a provider is configured.

Module 7 – Database, Project Identity and Versioning

SQLite persists project scans and their modules, health results, category scores, risk data, architecture scans, project identities, immutable version snapshots and file fingerprints. A stable project identity groups successive successful uploads as V1, V2, V3 and so on. Each version is associated with a scan; the version history is ordered and supports selection of an earlier scan for inspection.

Module 8 – Version Comparison and Code Churn

The comparison engine compares two stored versions of the same project. It reports added, removed, changed and unchanged file states; the unchanged state is verified using stored content hashes when available. It reports changes in supported stored metrics, project health/category-level values, risk signals and architecture/API-integration information where both versions have architecture results. Deterministic thresholds identify significant regressions or improvements.

Code Churn is not Git commit churn and is not an exact line diff. Added and removed files contribute their full LOC; a modified file contributes the absolute difference between its previous and current LOC when both values exist. Churn rate is calculated against previous-version LOC, and the score decreases as churn rate increases. Churn is unavailable for a first version or when the prior LOC baseline is not measurable.

Module 9 – Architecture Intelligence, Quality Gate and Language Health

Architecture Intelligence produces a structured architecture report from detected project structure and API/call relationships, including a generated diagram representation and coverage/uncertainty information. The Quality Gate evaluates stored metrics using advisory rules; the implementation explicitly notes that missing metrics are not treated as zero and that a historical-regression gate is not evaluated by that endpoint. Language Health provides a language-oriented comparison view over analyzed project data; it should be interpreted in the context of the supported static analyzers rather than as a controlled benchmark of language quality.

Module 10 – Dashboard and Historical Monitoring

The React dashboard provides project selection and version selection, health summary and category views, risk overview and risky-module details, AI recommendations, Architecture Intelligence, Quality Gate, Language Health, version comparison and historical health trends. Responsive styling is included in the frontend stylesheet. Historical trends are derived from saved version/scan records.

Module 11 – Reports and Project Deletion

The backend generates a PDF report from stored analysis information. The frontend SVG export combines the dashboard's already-rendered SVG health-gauge and history visualizations; CSS-based chart components are not redrawn into SVG. Project deletion is scoped to one selected logical project identity and removes its saved versions/scans and scan-owned modules, architecture results and fingerprints. Other project identities are not deleted.

12. Results and Performance

Application Screenshots :

The screenshots below document the AI Software Metrics Advisor interface, project upload and version handling, health dashboard, trends, version comparison, architecture intelligence, risk results, quality gate, and AI recommendations.

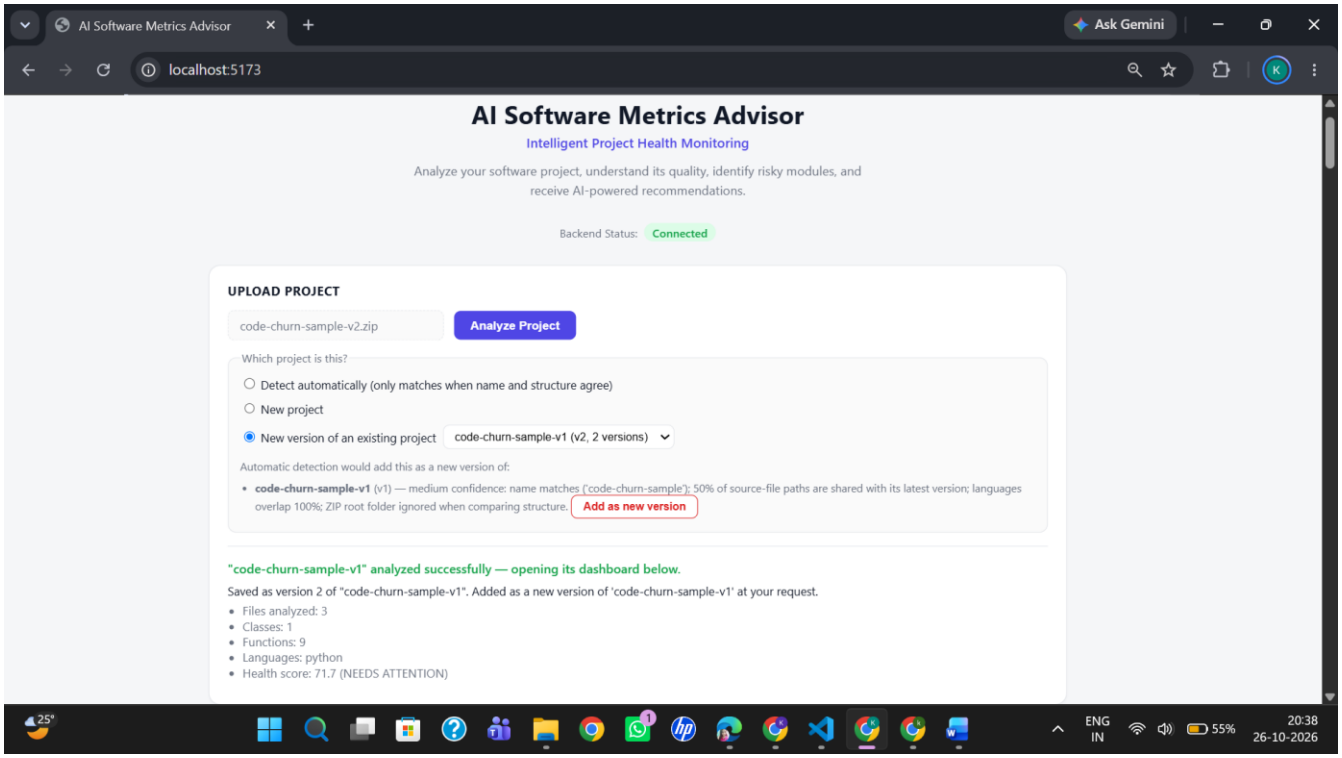


Figure A.1. Project upload and analysis result, including project/version selection and summary metrics.

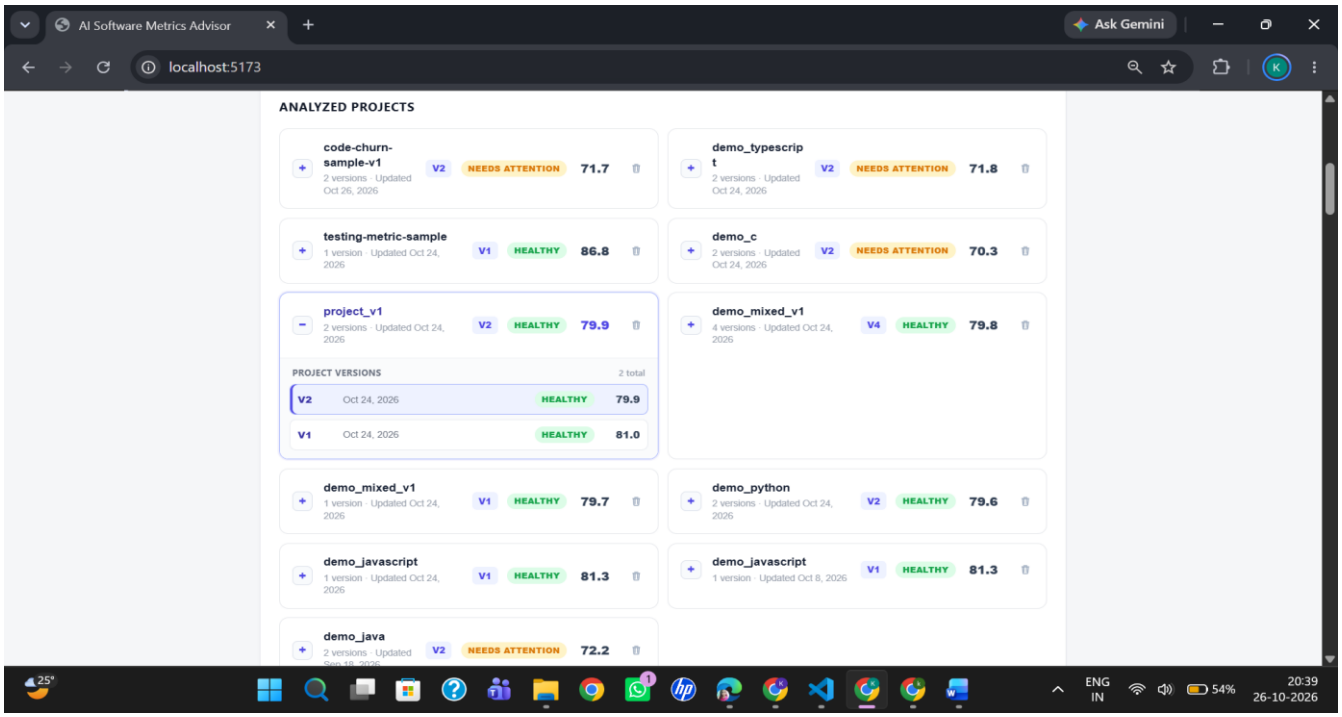


Figure A.2. Analyzed projects list and selected project dashboard controls.

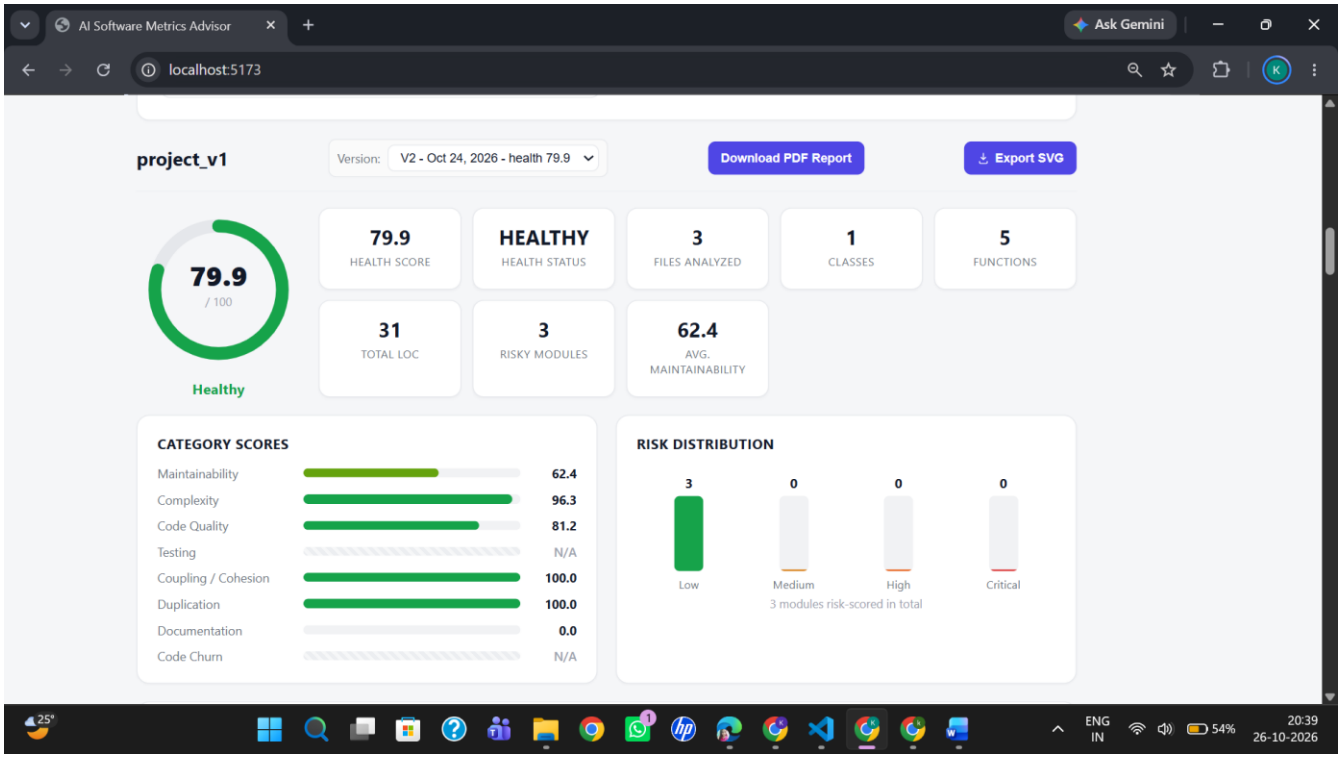


Figure A.3. Project health score, category scores, risk distribution, and project trends.

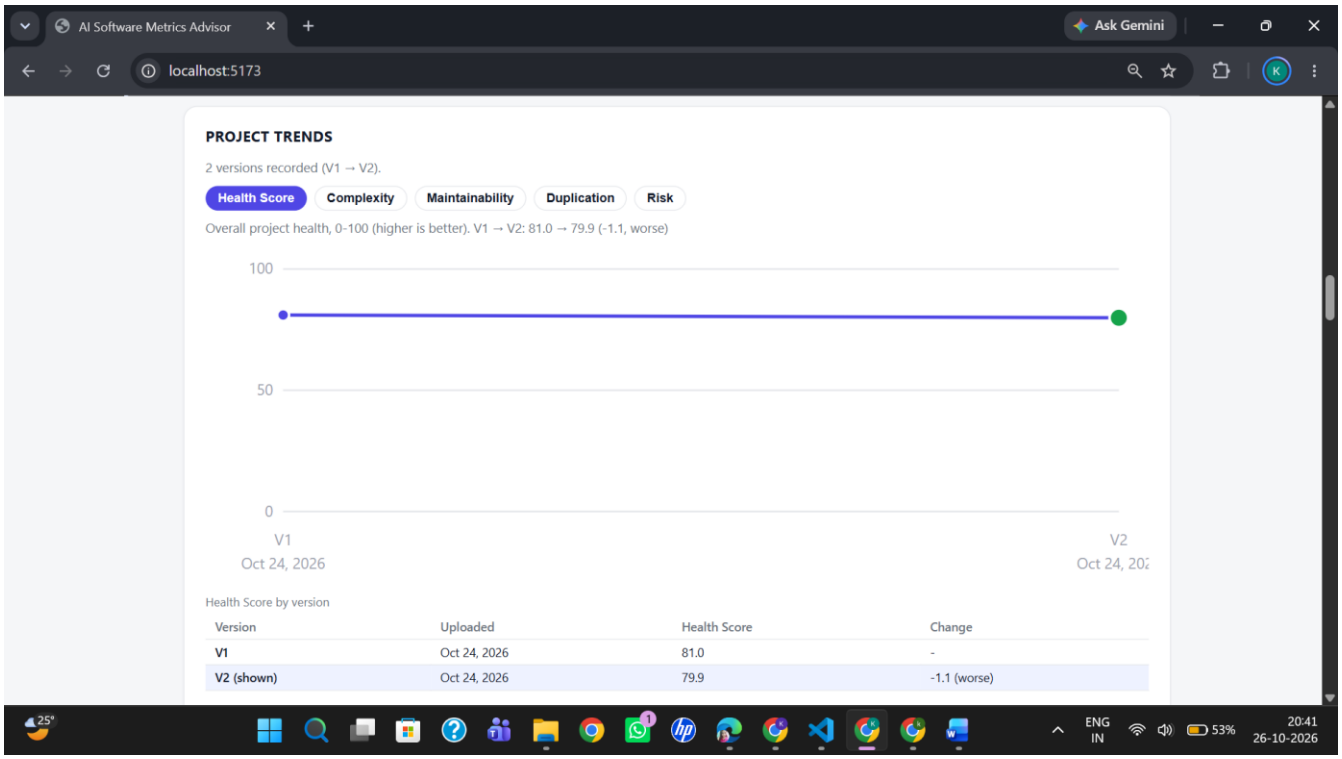


Figure A.4. Project trend chart and historical health-score table across versions.

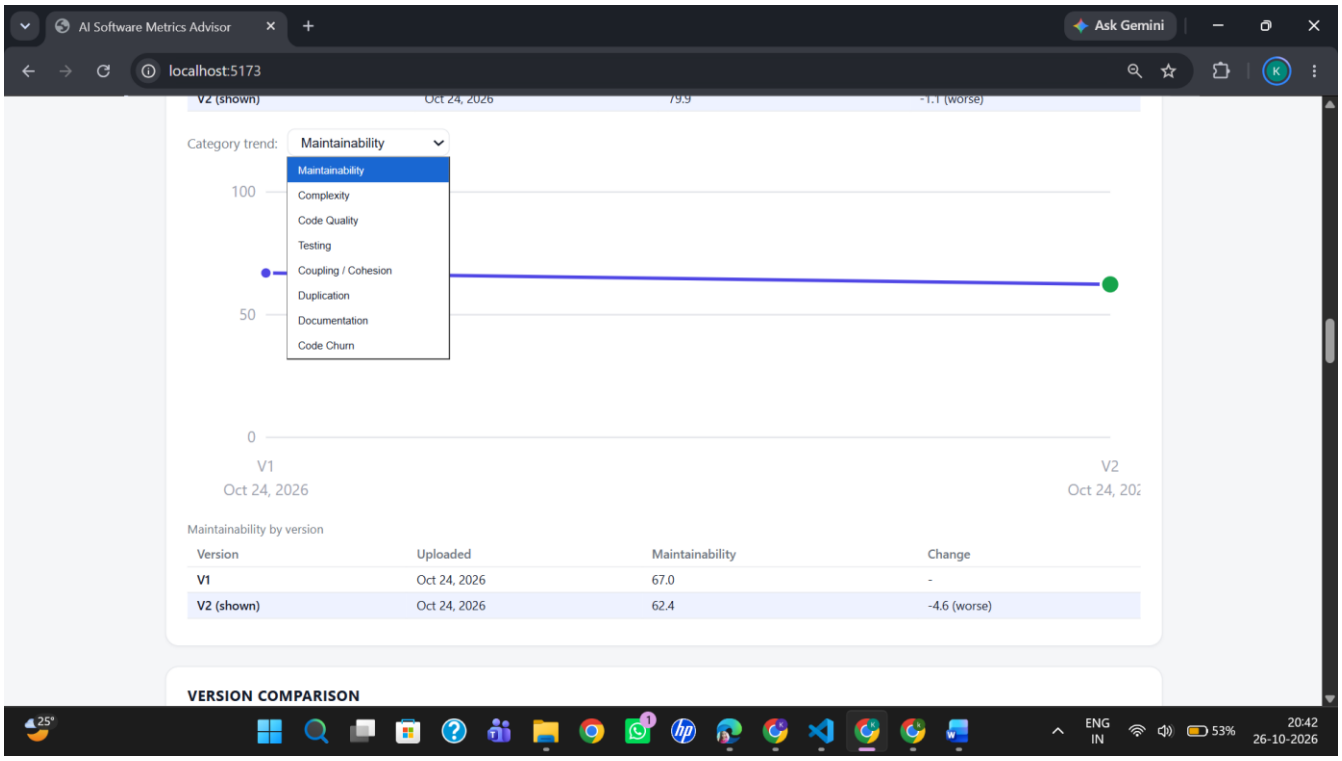


Figure A.5. Maintainability trend chart and version-wise maintainability values.

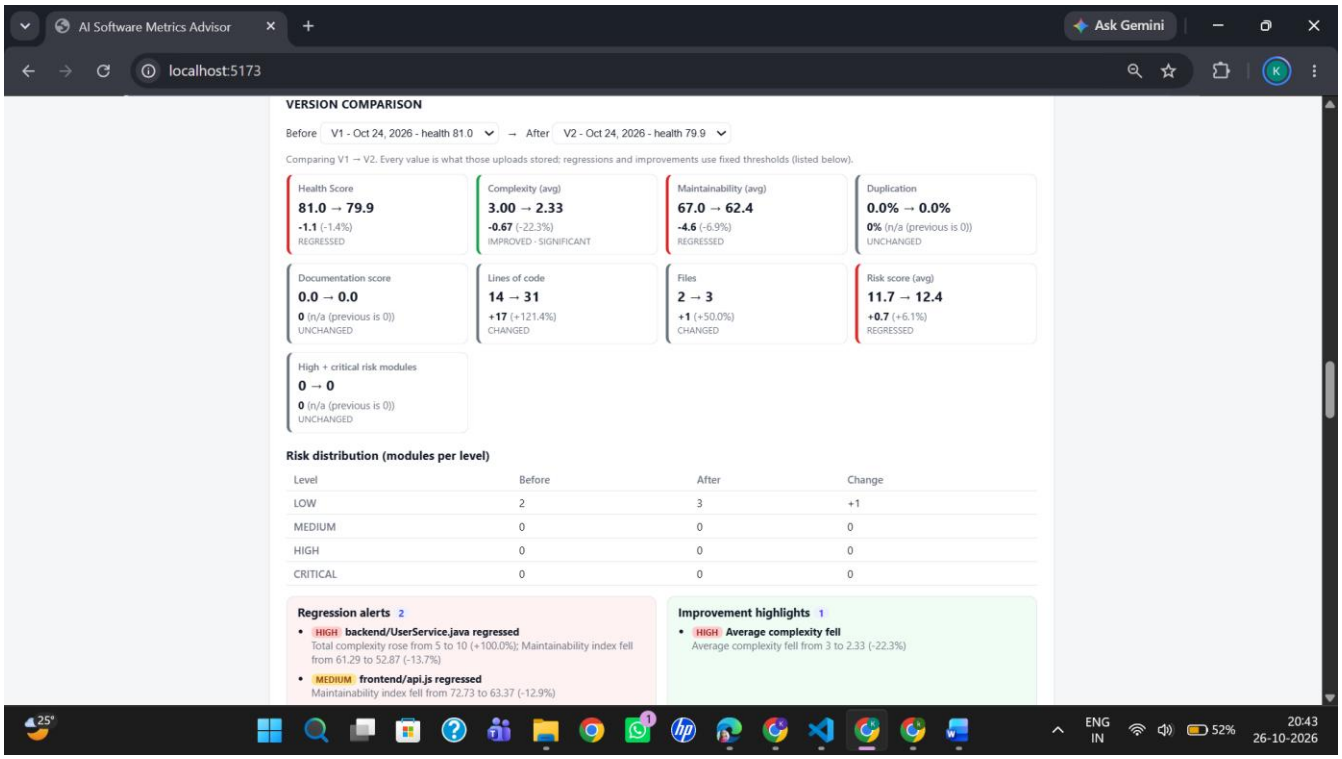


Figure A.6. Version comparison summary, risk distribution, regression alerts, and file-level comparison.

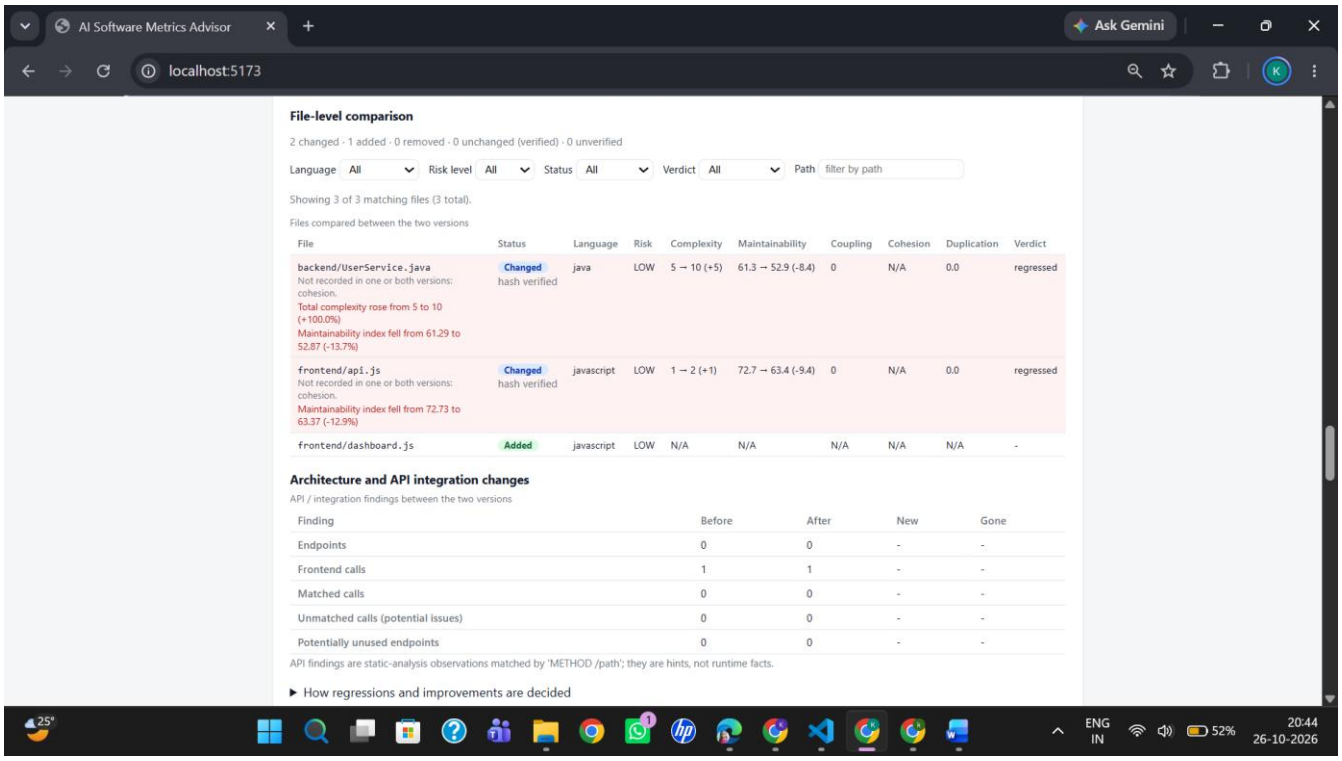


Figure A.7. File-level comparison and architecture/API integration findings.

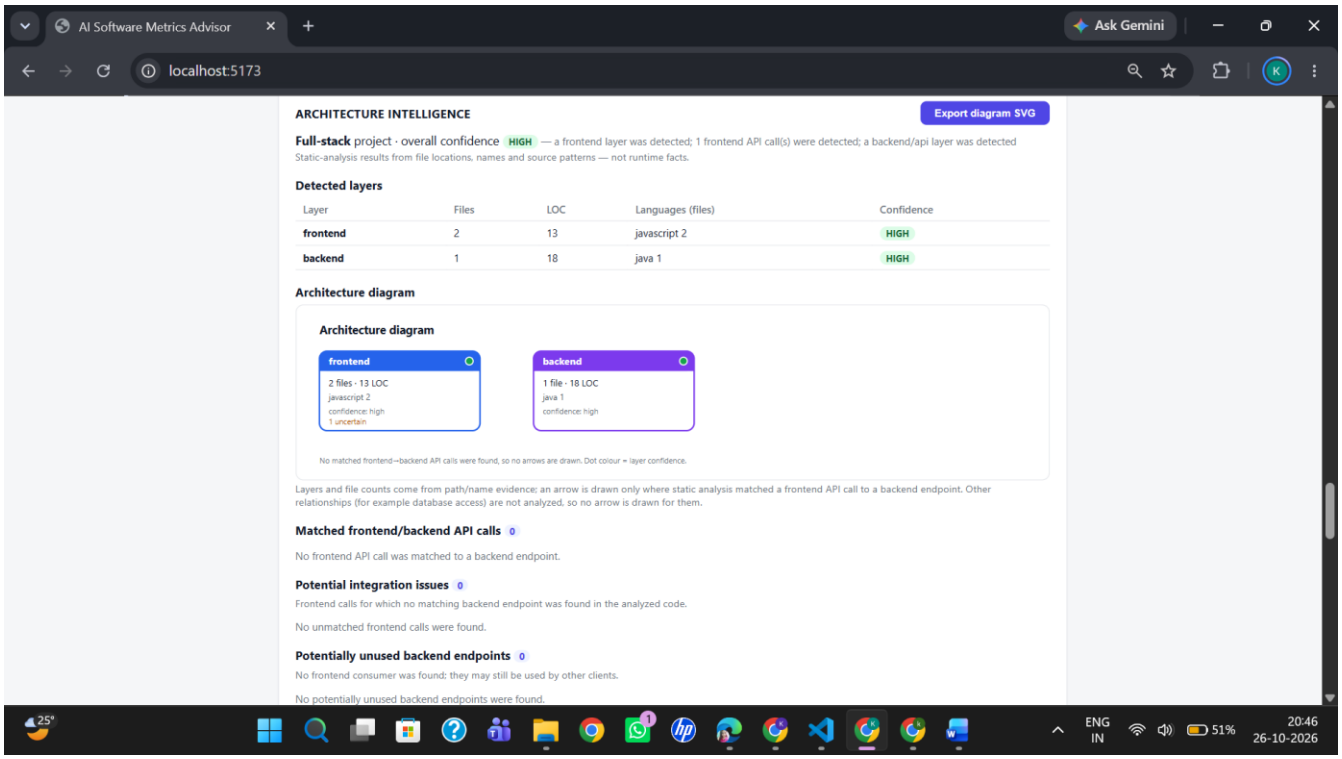


Figure A.8. Architecture intelligence, detected frontend/backend layers, and architecture diagram.

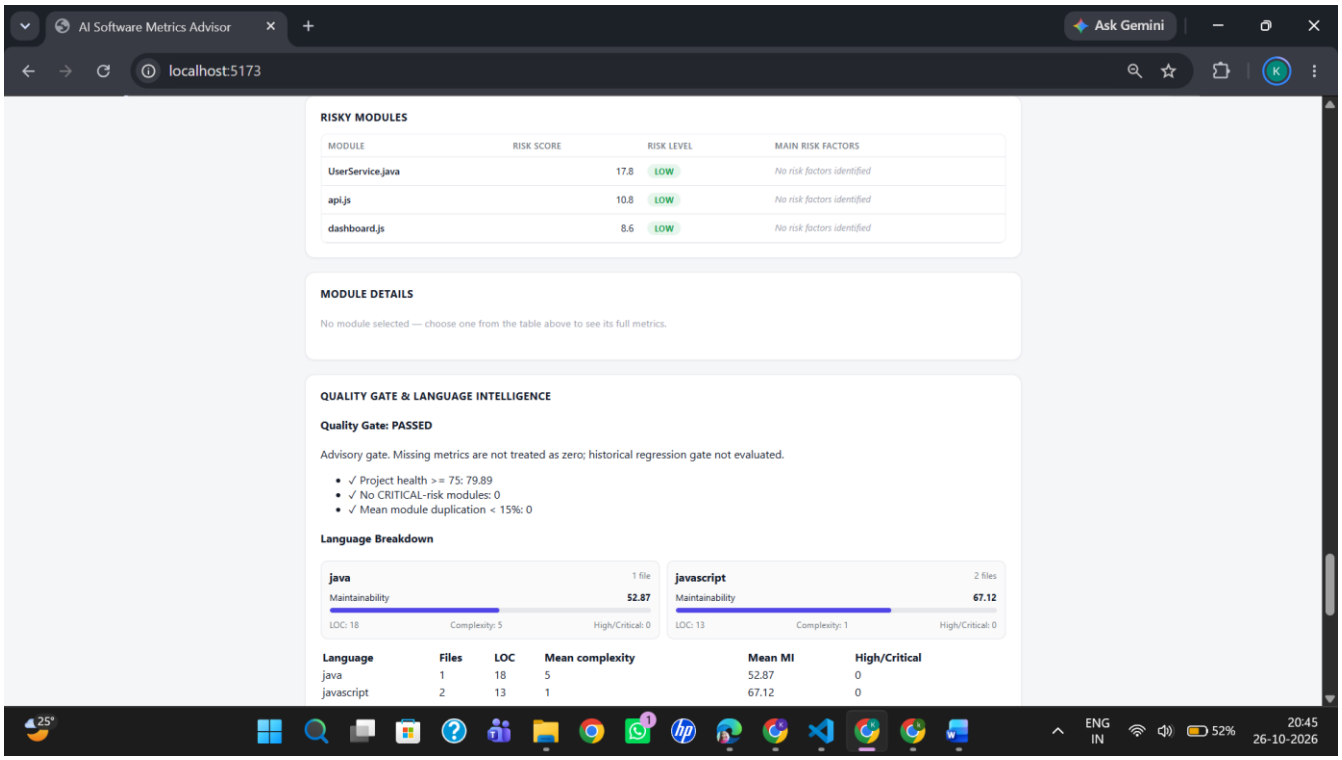


Figure A.9. Risky modules table, module details area, quality gate, and language intelligence.

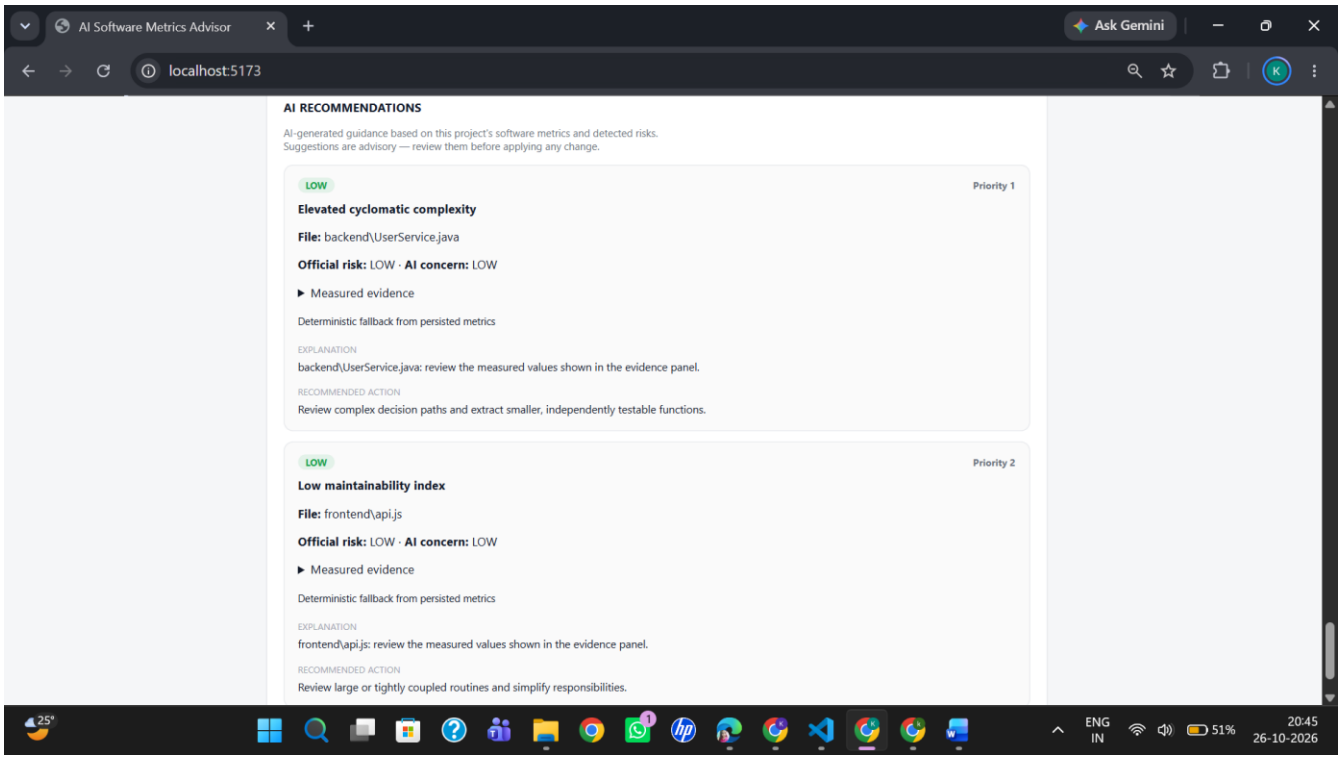


Figure A.10. AI-generated recommendations with risk labels, evidence, explanations, and suggested actions.

The following table combines previously documented demo results with the latest observed testing-metric and project_v1 version-comparison results supplied for this review. Results are sample observations and are not a universal benchmark.

Parameter	Result	Interpretation / scope
Demo Python Health	**79.58 – HEALTHY**	Observed sample result
Demo Java Health	**72.19 – NEEDS ATTENTION**	Observed sample result
Testing Sample Health	**86.8 – HEALTHY**	Quality Gate passed
Testing Score	**100.0**	Static test-file signal; not runtime coverage
V1 → V2 Health	**81.0 → 79.9**	Slight regression
Maintainability	**67.0 → 62.4**	Regression detected
Average Complexity	**3.00 → 2.33**	Improvement detected
Regression Alerts	**2 detected**	Issues identified in changed files
AI Recommendations	**Supported**	Recommendations are generated based on detected code-quality issues and metrics, helping identify areas that require attention.
Quality Gate	**PASSED**	No critical-risk modules
PDF / SVG Export	**Supported**	Reports and visualizations can be exported
Project Deletion	**Supported**	Project and associated stored versions/data can be deleted

The Testing metric is a static test-file presence signal based on recognized file and directory naming conventions; it is not runtime line or branch coverage. No accuracy percentage is reported because this deterministic static-metric workflow is not a supervised prediction model with a labeled accuracy evaluation. PDF/SVG generation time and project-deletion time were not measured. Code Churn is an LOC-based version-comparison proxy, not Git commit churn or an exact line diff. Missing measurements remain N/A rather than being treated as zero.

13. Comparison with Existing System

Feature	Conventional workflow targeted	Implemented proposed system
Metric calculation	May be distributed across tools or manual checks.	Integrated static analysis across supported languages.
Project-level health	Requires interpretation of individual metrics.	Category scores and weighted Project Health Score from available data.
Module risk	Manual prioritization or separate rules.	Deterministic module risk score, level and named factors.
AI recommendations	May require separate interpretation tools.	Optional AI Advisor uses stored metrics/risk evidence; does not replace scoring.
Project versioning	Can rely on names or external release notes.	Stable project identity groups immutable V1, V2, V3… versions.
File comparison	May require manual diff review.	Added/removed/changed/unchanged states, fingerprints and metric changes.
Regression/improvement	Manual assessment across reports.	Rule-based significance thresholds and direction signals.
Code Churn	May rely on Git statistics or manual diff.	Implemented LOC-based version comparison proxy; not Git commit statistics or exact line diff.
Historical monitoring	May require manually maintained records.	Saved version history and project-health timeline.
Architecture / decision support	May require separate tools.	Architecture Intelligence, advisory Quality Gate and Language Health views.
Reporting	Manual compilation may be needed.	Generated PDF report and supported SVG visualization export.
Project deletion	Depends on local workflow.	Deletion scoped to one logical project and its associated stored versions/data.

This table compares the implemented workflow with the conventional problem context described in the project. The comparison represents the conventional workflow targeted by this project and is not a controlled benchmark against commercial products.

14. Innovation Validation

What is the innovation?

The innovation is the integration of multi-metric static analysis, a unified health score, transparent module risk rules, optional evidence-grounded AI recommendations, and version-aware project monitoring in one interface.

What new feature or approach has been introduced?

The system links stored metrics and risks to project identity and immutable versions. It compares file status and metric changes between versions, derives an LOC-based churn signal, displays health history, and provides Architecture Intelligence, Quality Gate, Language Health, PDF reporting and SVG export.

Why is it innovative and how is it different?

The combined workflow reduces the need to assemble separate metric outputs manually. The AI layer is intentionally separated from deterministic analysis: it interprets evidence rather than inventing or recalculating health scores. Version comparison also distinguishes content-hash-verified unchanged files from files whose content cannot be verified.

What problem does it solve?

It helps a developer identify quality concerns at both project and module level and inspect how stored health and file-level measurements change across successive uploads.

How was it validated?

Validation uses documented sample outputs and stored version-comparison results. The demo Python and Java scores are 79.58 (HEALTHY) and 72.19 (NEEDS ATTENTION). The testing-metric sample records health 86.8 (HEALTHY), a Testing score of 100.0 based on recognized test files, and a PASSED advisory Quality Gate. For project_v1, health changed from 81.0 in V1 to 79.9 in V2; comparison identified two regression alerts (backend/UserService.java and frontend/api.js) and one improvement highlight, with average complexity decreasing from 3.00 to 2.33. File comparison reported two changed files, one added file, and zero removed or unchanged files. Language Health and Architecture Intelligence were evaluated using the supplied sample projects.. PDF report generation, supported SVG export, and project-scoped deletion are implemented, but their execution times were not measured. The ZIP contains tests for relevant implementation paths; presence of test files does not establish that every test passed, nor does it provide runtime code coverage.

Validation evidence	Recorded status / limitation
Sample Python and Java project outputs	Scores 79.58 (HEALTHY) and 72.19 (NEEDS ATTENTION) are documented in the existing review.
Upload and analysis timing	602.91 ms for the previously recorded demo Java upload/analysis run.
AI timing	15.84 seconds for the previously recorded local Ollama request; environment-dependent.
Automated test suite	Test files are present in the ZIP; full-suite pass count not asserted in this document.
Runtime code coverage	Not measured; uploaded source code is not executed by the analyzer.
Accuracy / predictive performance	Not applicable / not measured for the deterministic static-metric workflow.
Testing-metric sample	Health 86.8 HEALTHY; Testing score 100.0 based on recognized test files; runtime coverage not measured.
project v1 version comparison	Health 81.0 → 79.9 (−1.1); complexity 3.00 → 2.33; maintainability 67.0 → 62.4.
File-level comparison and regression	2 changed, 1 added, 0 removed, 0 unchanged; 2 regression alerts; 1 improvement highlight.
Quality Gate	PASSED for the supplied testing-metric sample and project v1 V2 results; advisory rule-based evaluation.
Architecture and Language Health	Frontend/backend/test structure and Java/JavaScript/Python language-oriented metrics recorded for the supplied samples.
PDF, SVG and project deletion	Supported; PDF/SVG/deletion timing not measured.

15. Future Enhancement

The following are possible enhancements rather than claims about current implementation:

- Integrate coverage reports from test runners to report actual line/branch coverage.
- Use Git history or exact source diffs for commit-based churn and changed-line statistics.
- Improve parser/type-resolution precision, especially for coupling, cohesion and cross-language project structures.
- Extend analysis to additional languages beyond the currently supported Python, Java, C, C++, JavaScript and TypeScript.
- Add CI/CD integration and configurable quality-gate enforcement in a development pipeline.
- Evaluate the system on a larger, documented benchmark set with repeatable performance measurements.
- Add team accounts, shared project access and collaboration workflows if required.
- Extend historical analytics and evaluate predictive risk models separately from deterministic risk rules.

16. Conclusion

The AI-Assisted Software Metrics Advisor with Intelligent Project Health Monitoring implements a unified workflow for static source analysis, software-quality scoring, deterministic module risk detection, optional AI-assisted recommendations, project versioning, comparison and historical monitoring. It analyzes supported source files without executing uploaded code and keeps unavailable metric values distinct from measured zero values.

The implementation includes stable project identities and immutable versions, file-level comparison using stored fingerprints, LOC-based Code Churn, Architecture Intelligence, an advisory Quality Gate, Language Health, project-scoped deletion, PDF reports and SVG export of supported dashboard charts. Recorded sample results include 79.58 (HEALTHY) for the demo Python project, 72.19 (NEEDS ATTENTION) for the demo Java project, and 86.8 (HEALTHY) for the testing-metric sample. For project_v1, V1 health of 81.0 changed to 79.9 in V2, with two regression alerts and an average-complexity improvement from 3.00 to 2.33. These are contextual observations rather than universal performance guarantees.

The main contribution is the integration of transparent static metrics and deterministic risks with a separate evidence-based AI interpretation layer and version-aware monitoring. Future evaluation should measure repeatability, broader project coverage, and actual runtime test coverage through external coverage reports before making stronger performance or coverage claims.